- Основы Stream API
  - Что такое Stream
  - Устройство конвейера
  - Функциональные интерфейсы и ссылки на методы
  - Ленивость и порядок выполнения
  - Источники стримов
- Промежуточные операции
  - Преобразование — map, flatMap, mapMulti
  - Фильтрация и срезы
  - Stateful-операции — distinct, sorted
  - peek — только для отладки
- Терминальные операции
  - Сбор в коллекцию — toList vs collect
  - reduce — свёртка
  - Поиск и проверки
  - forEach vs forEachOrdered
- Collectors
  - toMap
  - groupingBy и partitioningBy
  - joining, teeing и прочее
- Примитивные стримы
- Optional как результат стрима
- Parallel Streams
- Gatherers (Java 24+)
- Вопросы к собеседованию

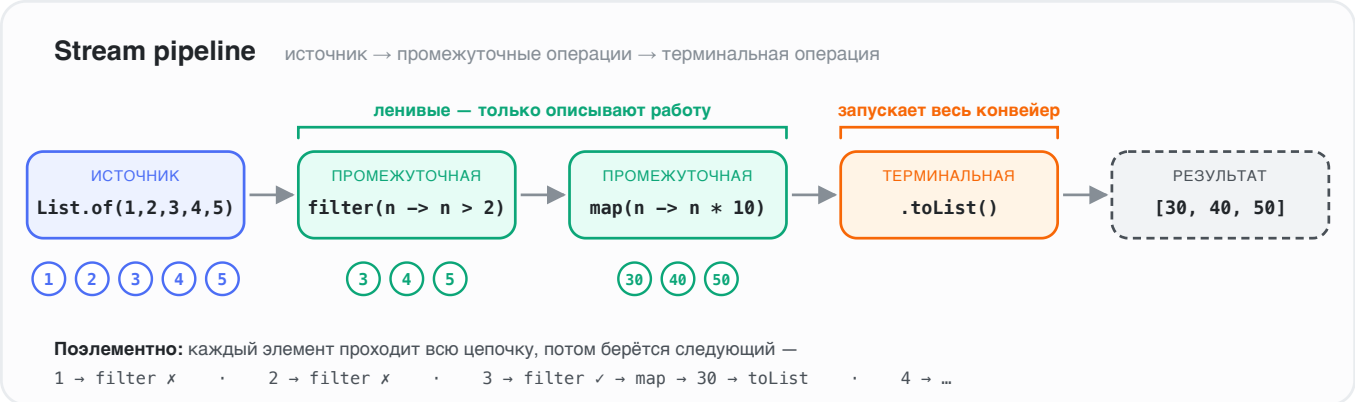
- Теория
- Код с собеседований

Основы Stream API

Что такое Stream

Stream (`java.util.stream.Stream<T>` , Java 8) — последовательность элементов, над которой декларативно описывается конвейер операций. Стрим **не хранит данные** (это не коллекция), **не изменяет источник** и **потребляется один раз**.

```
List<String> names = users.stream()
        .filter(u -> u.age() >= 18)
        .map(User::name)
        .sorted()
        .toList();
```



Ключевые свойства:

- **Ленивость** — промежуточные операции ничего не делают, пока не вызвана терминальная.

- **Одноразовость** — повторное использование бросает `IllegalStateException: stream has already been operated upon or closed`.
- **Внутренняя итерация** — цикл крутит сама библиотека, а не код (`for`), поэтому возможны оптимизации и параллелизм.

```
Stream<Integer> s = Stream.of(1, 2, 3);
s.count();
s.count();                                    //
IllegalStateException

Supplier<Stream<Integer>> src = () -> Stream.of(1, 2,
3);
src.get().count();                            // ✅
каждый раз новый стрим
src.get().count();
```

> ⚠ **Стрим — не всегда лучше цикла**
>
> Для обхода с ранним `return` / `break`, изменения нескольких переменных или работы с индексами обычный `for` читается проще.

Устройство конвейера

Pipeline = источник → 0..N промежуточных операций → 1 терминальная.

Часть	Что делает	Возвращает	Примеры
Источник	создаёт стрим	`Stream<T>`	`list.stream()`, `Stream.of()`
Промежуточная	описывает преобразование, лениво	`Stream<R>`	`filter`, `map`, `sorted`
Терминальная	запускает вычисление	результат / `void`	`toList`, `collect`, `cou`

Промежуточные операции делятся на:

- **stateless** — обрабатывают элемент независимо от других (`filter`, `map`);
- **stateful** — должны видеть другие элементы (`sorted`, `distinct`, `limit`) → буферизуют данные, хуже параллелятся.

> ⚠ **Лямбды без побочных эффектов**
>
> Лямбды в стриме должны только вычислять результат из элемента, а не менять внешние поля/коллекции. Иначе код ломается в parallel и зависит от порядка выполнения (см. Parallel Streams).

Short-circuiting — операции, которые могут завершиться, не обработав весь стрим: `limit`, `takeWhile`, `findFirst`, `anyMatch` и т.д. Только с ними можно безопасно работать с бесконечными стримами.

Функциональные интерфейсы и ссылки на методы

Функциональный интерфейс — интерфейс с одним абстрактным методом; его реализацию можно передать лямбдой или ссылкой на метод. Все операции стримов принимают именно их (`java.util.function`).

Интерфейс	Сигнатура	Где в стримах
`Predicate<T>`	`T → boolean`	`filter` , `anyMatch` , `takeWhile`
`Function<T, R>`	`T → R`	`map` , `flatMap` , `groupingBy` , `toMap`
`Consumer<T>`	`T → void`	`forEach` , `peek`
`Supplier<T>`	`() → T`	`generate` , `orElseGet` , `toCollection`
`UnaryOperator<T>`	`T → T`	`iterate`
`BinaryOperator<T>`	`(T, T) → T`	`reduce` , merge-функция в `toMap`
`BiConsumer<T, U>`	`(T, U) → void`	`mapMulti`
`Comparator<T>`	`(T, T) → int`	`sorted` , `min` , `max`

Для примитивов есть версии без боксинга: `IntPredicate` , `IntFunction` , `ToIntFunction` (для `mapToInt`) и т.д.

Ссылка на метод — короткая запись лямбды, которая только вызывает готовый метод:

```
Integer::parseInt      // статический метод          s
-> Integer.parseInt(s)
String::length         // метод элемента             s
-> s.length()
System.out::println    // метод конкретного объекта  x
-> System.out.println(x)
ArrayList::new         // конструктор                ()
-> new ArrayList<>()
```

Ленивость и порядок выполнения

Ленивость — промежуточные операции только описывают работу; выполняется всё в момент вызова терминальной операции.

```
Stream.of(1, 2, 3).map(n -> n * 10);   // ничего не
произошло — нет терминальной
```

Поэлементно, а не целиком. Стрим не делает «сначала `map` для всех, потом `forEach` для всех». Каждый элемент проходит **всю цепочку**, и только потом берётся следующий:

```
Stream.of(1, 2, 3)
      .map(n -> { System.out.println("map " + n);
return n; })
      .forEach(n -> System.out.println("forEach " +
n));
```

```
map 1
forEach 1
map 2
```

```
forEach 2
map 3
forEach 3
```

Цикл бы дал `map 1, map 2, map 3, forEach 1, …` — у стрима не так.

Отсюда short-circuit — лишние элементы вообще не трогаются:

```
Stream.of(1, 2, 3, 4)
      .filter(n -> { System.out.println("filter " + n);
return n > 1; })
      .findFirst();
```

```
filter 1
filter 2     ← найден → 3 и 4 не проверяются
```

> **Порядок операций влияет на скорость**
>
> Раз каждая операция выполняется для каждого дошедшего элемента — сначала отсекаем, потом считаем: `filter` → дорогой `map`; `filter` → `sorted`, а не наоборот.

Источники стримов

```
list.stream();                          // любая
Collection
Arrays.stream(arr);                     // массив (для
int[] → IntStream)
Stream.of(1, 2, 3);
Stream.ofNullable(value);               // 0 или 1
```

```
элемент (Java 9)
Stream.empty();
Stream.concat(s1, s2);

Stream.iterate(1, n -> n * 2).limit(10);          //
бесконечный + limit
Stream.iterate(1, n -> n < 1000, n -> n * 2);     // с
условием, как for (Java 9)
Stream.generate(UUID::randomUUID).limit(5);

IntStream.range(0, 10);                  // [0, 10)
IntStream.rangeClosed(1, 10);            // [1, 10]
"text".chars();                          // IntStream
кодов символов
map.entrySet().stream();
Files.lines(path);                       // держит файл
открытым → try-with-resources
```

```
try (Stream<String> lines =
Files.lines(Path.of("log.txt"))) {
    long errors = lines.filter(l ->
l.contains("ERROR")).count();
}
```

⚠ Бесконечные стримы

`iterate` / `generate` без условия бесконечны: нужна short-circuit операция (`limit` , `takeWhile` , `findFirst` , `anyMatch`). `Stream.generate(...).count()` или `.sorted()` — зависание.

⚠ **Не менять источник во время обхода**

```
List<Integer> list = new ArrayList<>(List.of(1, 2,
3));
list.stream().forEach(n -> list.add(n));   //
ConcurrentModificationException
```

Промежуточные операции

Возвращают новый `Stream` → их можно цеплять. Сами ничего не запускают.

Преобразование — map, flatMap, mapMulti

`map`

Что делает: превращает каждый элемент в новый — **1 → 1**.

```
Stream.of("a", "bb", "ccc")
      .map(String::length)
      .toList();                    // [1, 2, 3]
```

⚠ **Checked-исключения**

`Function` не объявляет `throws`, поэтому метод с checked-исключением в лямбду напрямую не передать:

```
paths.stream().map(Files::readString);
// ❌ не компилируется: IOException
paths.stream().map(p -> {
```

```
        try { return Files.readString(p); }
        catch (IOException e) { throw new
    UncheckedIOException(e); }
    });
    // ✅
```

`flatMap`

Что делает: превращает каждый элемент в **стрим** (0..N элементов) и склеивает их в один — **1 → много**.

```
Stream.of(List.of(1, 2), List.of(3))
      .flatMap(List::stream)
      .toList();                    // [1, 2, 3]

Stream.of("a b", "c")
      .flatMap(s -> Arrays.stream(s.split(" ")))
      .toList();                    // [a, b, c]
```

`map` здесь дал бы `[[1, 2], [3]]` — стрим списков, а не чисел.

`mapMulti` (Java 16)

Что делает: то же, что `flatMap`, но элементы отдаются в `sink` напрямую, без создания промежуточного стрима. Удобно, когда на выходе 0–2 элемента.

```
Stream.of(1, 2, 3)
      .<Integer>mapMulti((n, sink) -> {
          sink.accept(n);
          sink.accept(n);
```

```
        })
        .toList();                    // [1, 1, 2, 2, 3, 3]
```

Фильтрация и срезы

`filter`

Что делает: оставляет элементы, для которых условие `true`.

```
Stream.of(1, 2, 3, 4, 5)
        .filter(n -> n % 2 == 0)
        .toList();                    // [2, 4]
```

`limit`

Что делает: берёт первые N элементов и останавливает стрим. Главный способ ограничить бесконечный стрим.

```
Stream.of(1, 2, 3, 4, 5).limit(3).toList();
// [1, 2, 3]
Stream.iterate(1, n -> n * 2).limit(5).toList();
// [1, 2, 4, 8, 16]
```

`skip`

Что делает: пропускает первые N элементов.

```
Stream.of(1, 2, 3, 4, 5).skip(2).toList();
// [3, 4, 5]
```

`takeWhile` (Java 9)

Что делает: берёт элементы, **пока** условие `true`; на первом `false` — стоп.

```
Stream.of(1, 2, 3, 1).takeWhile(n -> n < 3).toList();
// [1, 2]
Stream.of(1, 2, 3, 1).filter(n -> n < 3).toList();
// [1, 2, 1] ← filter идёт до конца
```

`dropWhile` (Java 9)

Что делает: пропускает элементы, **пока** условие `true`; с первого `false` берёт всё остальное.

```
Stream.of(1, 2, 3, 1).dropWhile(n -> n < 3).toList();
// [3, 1]
```

Stateful-операции — distinct, sorted

Для работы им нужно видеть **другие элементы**, поэтому они накапливают данные в буфере. На бесконечном стриме не завершатся.

`distinct`

Что делает: убирает дубликаты (по `equals()` / `hashCode()`), порядок первого появления сохраняется.

```
Stream.of(1, 1, 2, 3, 3).distinct().toList();
// [1, 2, 3]
```

`sorted`

Что делает: сортирует — по естественному порядку (`Comparable`) или по `Comparator`.

```
Stream.of(3, 1, 2).sorted().toList();
// [1, 2, 3]
Stream.of(3, 1,
2).sorted(Comparator.reverseOrder()).toList();     //
[3, 2, 1]
Stream.of("ccc", "a", "bb")
      .sorted(Comparator.comparing(String::length))
      .toList();
// [a, bb, ccc]
```

Несколько критериев: `Comparator.comparing(User::age).thenComparing(User::name)`.

peek — только для отладки

`peek`

Что делает: выполняет действие над элементом и передаёт его дальше без изменений.

```
Stream.of(1, 2, 3)
      .peek(n -> System.out.println("до map: " + n))
      .map(n -> n * 10)
      .toList();                  // [10, 20, 30], в
консоли: до map: 1, 2, 3
```

Не использовать для логики: стрим может пропустить `peek`.

```
List.of(1, 2, 3).stream()
    .peek(System.out::println)
    .count();                          // Java 9+: ничего не
напечатает — размер известен заранее
```

Терминальные операции

Запускают конвейер и возвращают результат (не стрим). После них стрим использовать нельзя.

Сбор в коллекцию — toList vs collect

`toList` (Java 16)

Что делает: собирает элементы в **неизменяемый** `List`.

```
List<Integer> list = Stream.of(1, 2, 3).toList();
// [1, 2, 3]
list.add(4);
// UnsupportedOperationException
```

`collect`

Что делает: собирает элементы с помощью `Collector` — в любую коллекцию, `Map` или строку (см. Collectors).

```
Stream.of(1, 2, 2).collect(Collectors.toSet());
// [1, 2]
Stream.of(3,
1).collect(Collectors.toCollection(TreeSet::new));  //
[1, 3]
```

```
Stream.of(1, 2).collect(Collectors.toList());
// изменяемый (по факту ArrayList)
```

`toArray`

Что делает: собирает в массив.

```
Integer[] arr = Stream.of(1, 2,
3).toArray(Integer[]::new);
```

Способ	Изменяемость
`stream.toList()`	неизменяемый
`collect(Collectors.toList())`	не гарантируется (по факту `ArrayList`)
`collect(Collectors.toUnmodifiableList())`	неизменяемый

Современный дефолт — `.toList()`; `Collectors.toList()` — если список дальше нужно менять.

reduce — свёртка

`reduce`

Что делает: сводит все элементы к одному значению, применяя функцию попарно: `((1 + 2) + 3) + 4`.

```
Stream.of(1, 2, 3, 4).reduce(0, Integer::sum);      //
10  — с начальным значением
```

```
Stream.of(1, 2, 3, 4).reduce(Integer::max);           //
Optional[4] — без него → Optional
Stream.<Integer>empty().reduce(0, Integer::sum);      //
0
```

Функция должна быть **ассоциативной**, а начальное значение — нейтральным (`0` для `+`, `1` для `*`), иначе в parallel будет неверный результат.

Для сумм проще готовое: `mapToInt(...).sum()`.

Поиск и проверки

`count`

Что делает: возвращает количество элементов (`long`).

```
Stream.of(1, 2, 3).filter(n -> n > 1).count();
// 2
```

`min` / `max`

Что делает: находит минимум / максимум по `Comparator`.
Возвращает `Optional` — стрим может быть пустым.

```
Stream.of(3, 1, 2).max(Integer::compare);
// Optional[3]
Stream.of("a", "ccc",
"bb").min(Comparator.comparing(String::length)); //
Optional[a]
```

`findFirst` / `findAny`

Что делает: возвращает первый / любой элемент в `Optional`. `findAny` быстрее в parallel, но результат не детерминирован.

```
Stream.of(5, 6, 7).filter(n -> n > 5).findFirst();
// Optional[6]
Stream.<Integer>empty().findFirst();
// Optional.empty
```

`anyMatch` / `allMatch` / `noneMatch`

Что делает: проверяет условие — хотя бы один / все / ни один. Возвращает `boolean`, останавливается, как только ответ ясен.

```
Stream.of(1, 2, 3).anyMatch(n -> n > 2);
// true
Stream.of(1, 2, 3).allMatch(n -> n > 0);
// true
Stream.of(1, 2, 3).noneMatch(n -> n > 5);
// true

Stream.empty().allMatch(x -> false);
// true  ← на пустом стриме
Stream.empty().anyMatch(x -> true);
// false
```

forEach vs forEachOrdered

`forEach`

Что делает: выполняет действие для каждого элемента. В parallel порядок **не гарантирован**.

`forEachOrdered`

Что делает: то же, но всегда в порядке источника.

```
List.of(1, 2,
3).parallelStream().forEach(System.out::print);
// например 231
List.of(1, 2,
3).parallelStream().forEachOrdered(System.out::print);
// 123
```

Если нужно собрать результат — `toList()` / `collect`, а не `forEach` с `list.add(...)`.

Collectors

`Collector` — рецепт для `collect`: во что собирать (`supplier`), как добавлять элемент (`accumulator`), как сливать части в parallel (`combiner`), как получить итог (`finisher`). Готовые лежат в `java.util.stream.Collectors`.

toMap

`toMap`

Что делает: собирает в `Map`: первая функция → ключ, вторая → значение.

```
Stream.of("a", "bb", "ccc")
      .collect(Collectors.toMap(String::length, s ->
s));        // {1=a, 2=bb, 3=ccc}
```

Одинаковые ключи → `IllegalStateException`. Решается третьим аргументом — что делать с дублем:

```
Stream.of("a", "b", "cc")
      .collect(Collectors.toMap(String::length, s -> s,
(x, y) -> x + y)); // {1=ab, 2=cc}
```

Четвёртый аргумент — реализация `Map` (`TreeMap::new`).
Значение `null` → `NullPointerException`.

groupingBy и partitioningBy

`groupingBy`

Что делает: раскладывает элементы по группам: ключ → `List` элементов с этим ключом.

```
Stream.of("a", "bb", "cc", "ddd")
      .collect(Collectors.groupingBy(String::length));
// {1=[a], 2=[bb, cc], 3=[ddd]}
```

Второй аргумент — **что сделать с каждой группой** (downstream-коллектор):

```
// сколько в группе
.collect(Collectors.groupingBy(String::length,
Collectors.counting()));
// {1=1, 2=2, 3=1}

// преобразовать элементы группы
.collect(Collectors.groupingBy(String::length,
        Collectors.mapping(String::toUpperCase,
```

```
Collectors.toList())));
// {1=[A], 2=[BB, CC], 3=[DDD]}

// склеить группу в строку
.collect(Collectors.groupingBy(String::length,
Collectors.joining("+")));
// {1=a, 2=bb+cc, 3=ddd}
```

Другие downstream: `summingInt`, `averagingInt`, `maxBy`, `filtering` (9), `flatMapping` (9), `collectingAndThen`. Отсортированные ключи — `groupingBy(f, TreeMap::new, downstream)`.

`partitioningBy`

Что делает: делит на две группы по условию — всегда ключи `false` и `true`.

```
Stream.of(1, 2, 3, 4, 5)
      .collect(Collectors.partitioningBy(n -> n > 2));
// {false=[1, 2], true=[3, 4, 5]}
```

joining, teeing и прочее

`joining`

Что делает: склеивает строки — с разделителем, префиксом и суффиксом.

```
Stream.of("a", "b", "c").collect(Collectors.joining());
// abc
Stream.of("a", "b", "c").collect(Collectors.joining(",
", "[", "]"));  // [a, b, c]
```

`teeing` (Java 12)

Что делает: прогоняет элементы через **два** коллектора за один проход и объединяет их результаты.

```
double avg = Stream.of(1, 2, 3,
4).collect(Collectors.teeing(
        Collectors.counting(),                  // 4
        Collectors.summingInt(n -> n),          // 10
        (cnt, sum) -> (double) sum / cnt));     // 2.5
```

`summarizingInt`

Что делает: считает сразу count, sum, min, max, average.

```
IntSummaryStatistics st = Stream.of(1, 2,
3).collect(Collectors.summarizingInt(n -> n));
// IntSummaryStatistics{count=3, sum=6, min=1,
average=2.000000, max=3}
```

`collectingAndThen`

Что делает: применяет функцию к результату другого коллектора.

```
Stream.of(1, 2,
3).collect(Collectors.collectingAndThen(Collectors.toLi
st(), List::size)); // 3
```

Примитивные стримы

`IntStream`, **`LongStream`**, **`DoubleStream`** — стримы примитивов без автобоксинга. Дают числовые терминальные операции.

```
int total      =
orders.stream().mapToInt(Order::qty).sum();
OptionalDouble avg = IntStream.of(1, 2, 3).average();
// OptionalDouble, не Optional<Double>
OptionalInt max    = IntStream.of(1, 2, 3).max();

IntStream.range(0, 5).boxed().toList();
// IntStream → Stream<Integer>
IntStream.range(0, 5).mapToObj(i -> "item" + i);
// → Stream<String>
```

Переходы: `mapToInt / mapToLong / mapToDouble / mapToObj / boxed / asLongStream`.

`Stream<Integer>` вместо `IntStream` в горячем коде = лишние объекты `Integer` на каждый элемент.

Optional как результат стрима

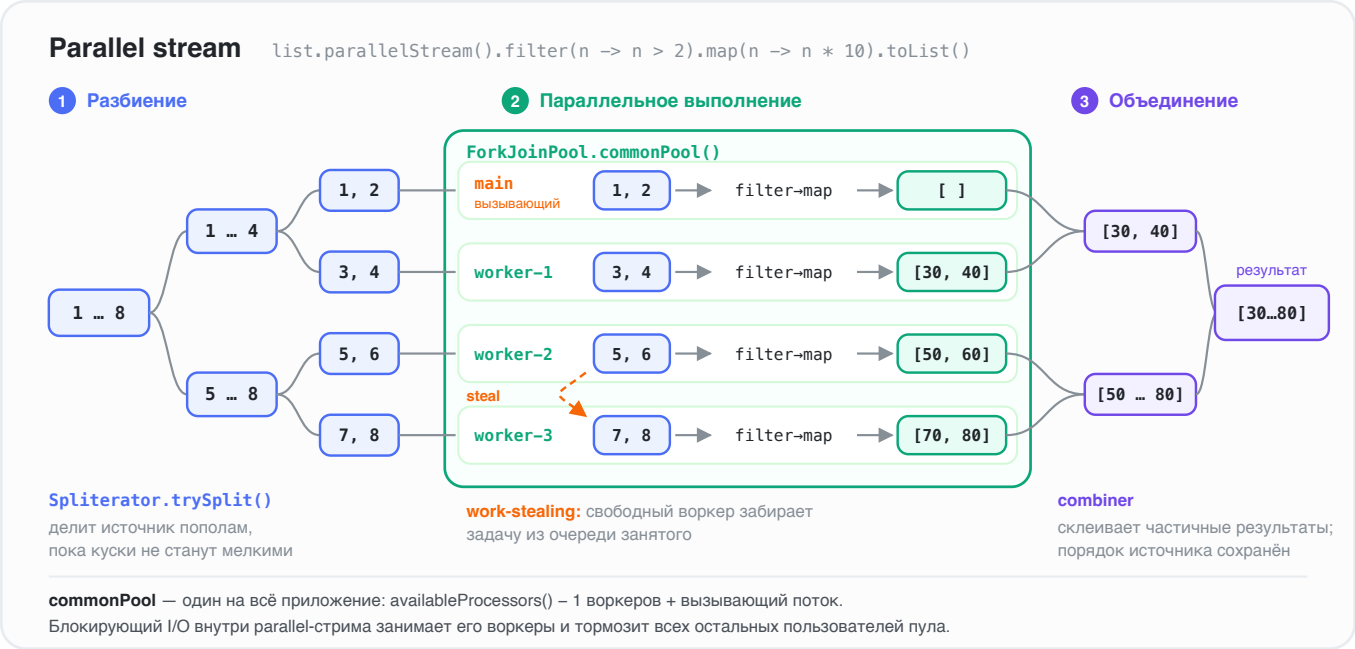
Терминальные операции, которые могут не найти элемент (`findFirst`, `min`, `max`, `reduce` без identity), возвращают **`Optional<T>`**.

```
String name = users.stream()
        .filter(User::isAdmin)
        .map(User::name)
        .findFirst()
```

```
        .orElse("none");          //
orElseGet(supplier), orElseThrow()
```

`Optional::stream` (Java 9) превращает `Optional` в стрим из 0/1 элемента — удобно во `flatMap`.

Parallel Streams



Параллельный стрим — стрим, который `Spliterator` делит на части, обрабатываемые в `ForkJoinPool.commonPool()` (по умолчанию `availableProcessors() – 1` воркеров + вызывающий поток).

```
long count =
bigList.parallelStream().filter(this::isPrime).count();
stream.parallel();   // переключить; .sequential() —
обратно (действует на весь конвейер)
```

Когда имеет смысл:

- данных много, и работа **CPU-bound**;
- источник хорошо делится: `ArrayList`, массивы, `IntStream.range` — да; `LinkedList`, `Stream.iterate`, `Files.lines` — плохо;
- операции stateless, без общего изменяемого состояния.

Когда не надо:

- I/O и блокирующие вызовы — забивают общий `commonPool`, мешая всем остальным его пользователям (включая `CompletableFuture` по умолчанию);
- маленькие коллекции — накладные расходы на разбиение больше выигрыша;
- `sorted`, `limit`, `findFirst`, `forEachOrdered` на упорядоченных стримах — порядок дорого сохранять (`unordered()` может помочь).

Гонка данных в parallel:

```
List<Integer> res = new ArrayList<>();
nums.parallelStream().forEach(res::add);      // ❌ потеря элементов / исключение
List<Integer> ok = nums.parallelStream().toList(); // ✅
```

Gatherers (Java 24+)

Gatherer (`java.util.stream.Gatherer`, финализирован в Java 24, JEP 485) — пользовательская **промежуточная** операция, как `Collector` для терминальных. Подключается через

`stream.gather(...)`. Закрывает то, что раньше не делалось стримами: окна, накопление с состоянием.

```
import java.util.stream.Gatherers;

Stream.of(1, 2, 3, 4,
5).gather(Gatherers.windowFixed(2)).toList();
// [[1, 2], [3, 4], [5]]

Stream.of(1, 2, 3,
4).gather(Gatherers.windowSliding(3)).toList();
// [[1, 2, 3], [2, 3, 4]]

Stream.of(1, 2, 3, 4).gather(Gatherers.scan(() -> 0,
Integer::sum)).toList();
// [1, 3, 6, 10] — нарастающий итог

urls.stream()
    .gather(Gatherers.mapConcurrent(10, this::fetch))
// до 10 параллельно, на виртуальных потоках
    .toList();
// порядок сохраняется
```

Встроенные: `windowFixed`, `windowSliding`, `fold`, `scan`, `mapConcurrent`. Свой gatherer — через `Gatherer.of(...)` / `Gatherer.ofSequential(...)`.

Вопросы к собеседованию

Теория

Чем Stream отличается от коллекции?

Коллекция **хранит** элементы, её можно обходить сколько угодно раз. Стрим **ничего не хранит**: он описывает вычисление над источником, выполняется лениво и только один раз. Коллекция — про данные, стрим — про обработку.

Подробнее: Что такое Stream.

Чем промежуточные операции отличаются от терминальных?

Промежуточные (`filter` , `map` , `sorted`) возвращают `Stream` и ничего не выполняют — только достраивают конвейер. Терминальная (`toList` , `collect` , `count` , `forEach`) запускает весь конвейер и возвращает результат. Без терминальной операции код в лямбдах не вызовется ни разу.

Подробнее: Устройство конвейера, Ленивость и порядок выполнения.

В чём разница между `map` и `flatMap` ?

`map` — один элемент в один (`1 → 1`). `flatMap` — один элемент в стрим из 0..N элементов, которые склеиваются в общий стрим (`1 → много`). Типичный случай — «распрямить» вложенные коллекции: `Stream<List<T>>` → `Stream<T>` .

```
Stream.of("ab", "cd").map(s -> s.split(""));
// Stream<String[]>
Stream.of("ab", "cd").flatMap(s ->
Arrays.stream(s.split(""))).toList(); // [a, b, c, d]
```

Подробнее: Преобразование — map, flatMap, mapMulti.

Почему стрим нельзя использовать повторно?

Стрим — это одноразовый конвейер, связанный с итератором (`Spliterator`) источника; после терминальной операции он израсходован. Повторный вызов → `IllegalStateException` . Если нужен ещё раз — создать новый (`list.stream()` или `Supplier<Stream<T>>`).

Что такое stateless / stateful и short-circuit операции?

- **stateless** — элемент обрабатывается независимо от других: `filter` , `map` .
- **stateful** — нужно видеть другие элементы, поэтому буферизуют: `sorted` , `distinct` . Дороже, хуже параллелятся, не работают с бесконечными стримами.
- **short-circuit** — могут закончить, не обработав всё: `limit` , `takeWhile` , `findFirst` , `anyMatch` .

`findFirst` vs `findAny` ?

`findFirst` возвращает первый элемент в порядке источника. `findAny` — любой; в последовательном стриме обычно тоже первый, но в parallel — первый, до которого добрался любой поток. Поэтому в parallel `findAny` быстрее: не нужно соблюдать порядок.

`Stream.toList()` vs `Collectors.toList()` ?

`toList()` (Java 16) возвращает неизменяемый список. `Collectors.toList()` не даёт гарантий о типе и изменяемости,

на практике — изменяемый `ArrayList`. `Collectors.toUnmodifiableList()` тоже неизменяемый, но запрещает `null`.

Подробнее: Сбор в коллекцию — toList vs collect.

`reduce` vs `collect`?

`reduce` — неизменяемая свёртка: каждый шаг создаёт новое значение (`(a, b) -> a + b`), подходит для чисел и неизменяемых объектов. `collect` — изменяемая свёртка: элементы добавляются в контейнер (`ArrayList`, `StringBuilder`, `HashMap`). Собирать коллекцию через `reduce` — значит создавать новую коллекцию на каждом шаге.

Почему лямбды в стриме не должны менять внешнее состояние?

Локальные переменные в лямбде должны быть effectively final — компилятор не даст их изменить. Менять поля и внешние коллекции компилятор позволит, но это побочный эффект: в parallel будет гонка данных, а из-за ленивости и short-circuit неизвестно, сколько раз и когда вызовется лямбда.

Подробнее: Устройство конвейера, Parallel Streams.

Почему `peek` нельзя использовать для логики?

Стрим вправе не прогонять элементы через операции, которые не влияют на результат. Например, если размер известен заранее, `count()` не выполнит `peek` (Java 9+). `peek` — только для отладки.

```
Stream.of("a", "b",
"c").peek(System.out::print).count();
// ничего
Stream.of("a", "b", "c").filter(s ->
true).peek(System.out::print).count();  // abc
```

Как работает parallel stream и когда он медленнее обычного?

`Spliterator` делит источник на куски, они выполняются в общем `ForkJoinPool.commonPool()`, частичные результаты объединяет `combiner`. Медленнее, когда данных мало, источник плохо делится (`LinkedList`, `iterate`), есть `sorted` / `limit` / `findFirst` с соблюдением порядка, есть боксинг или блокирующий I/O.

Запустить в своём пуле можно трюком `customPool.submit(() -> list.parallelStream()...).get()` — задачи пойдут в пул, из которого вызван стрим, но спецификация это не гарантирует.

Подробнее: Parallel Streams.

`orElse` vs `orElseGet`?

Аргумент `orElse` вычисляется **всегда**, даже если значение есть. `orElseGet` принимает `Supplier` и вызывает его, только когда `Optional` пуст. Для дорогих значений (запрос в БД, создание объекта) — `orElseGet`.

```
opt.orElse(loadDefault());       // loadDefault()
вызовется в любом случае
opt.orElseGet(this::loadDefault); // только если opt
пустой
```

Что такое Gatherer?

Пользовательская промежуточная операция (Java 24), аналог `Collector` для середины конвейера. Подключается через `.gather(...)`; готовые — окна (`windowFixed`, `windowSliding`), `scan`, `fold`, `mapConcurrent`.

Подробнее: Gatherers (Java 24+).

Код с собеседований

Частота слов

```
Map<String, Long> freq = Arrays.stream("java stream
java api stream java".split(" "))

.collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()));
// {java=3, stream=2, api=1}   (порядок HashMap не
гарантирован)
```

Самое частое слово

```
String top = freq.entrySet().stream()
        .max(Map.Entry.comparingByValue())
        .map(Map.Entry::getKey)
        .orElseThrow();   // java
```

Отсортировать Map по значению (по убыванию)

```
Map<String, Integer> m = Map.of("a", 3, "b", 1, "c",
2);

Map<String, Integer> sorted = m.entrySet().stream()
        .sorted(Map.Entry.<String,
Integer>comparingByValue().reversed())
        .collect(Collectors.toMap(Map.Entry::getKey,
Map.Entry::getValue,
                                  (x, y) -> x,
LinkedHashMap::new));  // LinkedHashMap — хранит
порядок
// {a=3, c=2, b=1}
```

Первый неповторяющийся символ

```
Optional<Character> first = "swiss".chars()
        .mapToObj(c -> (char) c)

.collect(Collectors.groupingBy(Function.identity(),
LinkedHashMap::new, Collectors.counting()))
        .entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey)
        .findFirst();     // Optional[w]
```

Дубликаты в списке

```
List<Integer> dups = Stream.of(1, 2, 3, 2, 4, 1)

.collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()))
        .entrySet().stream()
```

```
        .filter(e -> e.getValue() > 1)
        .map(Map.Entry::getKey)
        .toList();          // [1, 2]
```

Второе по величине число

```
Optional<Integer> second = Stream.of(5, 1, 5, 3, 4)
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst();       // Optional[4]
```

Сумма квадратов чётных чисел

```
int sum = IntStream.rangeClosed(1, 6)
        .filter(n -> n % 2 == 0)
        .map(n -> n * n)
        .sum();             // 4 + 16 + 36 = 56
```

Сотрудник с максимальной зарплатой в каждом отделе

```
record Emp(String name, String dept, int salary) {}

Map<String, String> topByDept = emps.stream()
        .collect(Collectors.groupingBy(Emp::dept,
                 Collectors.collectingAndThen(

Collectors.maxBy(Comparator.comparingInt(Emp::salary)),
                         e ->
```

```
e.orElseThrow().name())));
// {IT=Ann, HR=Eve}
```